

Observability & Monitoring — Practical System Design Notes

Hinglish • Logs • Metrics • Traces • p95/p99 • Prometheus • Grafana • OpenTelemetry • Debugging

1. Observability Kya Hai?

Observability ka goal hai system ke external behavior ko dekhkar internal state aur failures ko samajhna. Simple language mein: production mein system ke andar kya ho raha hai, uska evidence collect karke answer nikalna.

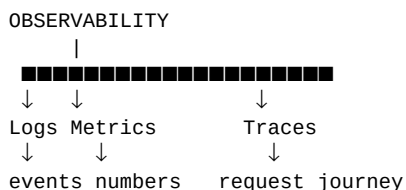
User says:
"API slow hai"

Observability helps answer:
Which API?
How slow?
Since when?
Which users?
CPU high?
DB slow?
Redis slow?
External API slow?
Queue backed up?

2. Monitoring vs Observability

Monitoring	Observability
Known conditions detect	Unknown problems investigate
Dashboards/alerts	Logs + metrics + traces + context
Is CPU > 80%?	Why did latency increase?
Often threshold based	Exploration and correlation

3. Three Pillars



Modern systems mein logs, metrics aur distributed traces ko correlation IDs/trace IDs ke through connect karna powerful pattern hai.

4. Logs

Logs discrete events ka record hain: request received, payment failed, DB timeout, worker retry etc.

```
{
  "level": "error",
  "service": "payment",
  "requestId": "abc123",
  "route": "/payments",
  "error": "timeout",
  "durationMs": 2100
}
```

Structured JSON logs search/filter/correlation ke liye plain text logs se generally easier hote hain.

5. What to Log

- Request/correlation ID.
- Service and environment.
- Route/method.
- Status code.
- Duration.
- Error type/code.
- Important business event ID.
- Dependency name.

Passwords, access tokens, card secrets, unnecessary PII etc. logs mein nahi daalne chahiye.

6. Metrics

Metrics numeric time-series data hain jo trends aur alerting ke liye useful hote hain.

```
http_requests_total
http_request_duration_seconds
process_cpu_usage
nodejs_heap_used_bytes
db_query_duration
queue_depth
```

7. Counters, Gauges, Histograms

Type	Use
Counter	Only increases: requests, errors, processed jobs
Gauge	Current value: memory, queue depth, active connections
Histogram	Distribution: request latency, DB latency

8. Traces

```
User Request
  ↓
Gateway
  ↓
Order Service
  ↓
Payment Service
  ↓
Database
```

Trace ID:
abc123

Each step = span

Distributed trace ek request ka end-to-end journey dikhata hai aur latency breakdown identify karne mein help karta hai.

9. Span

Trace ka individual operation span hota hai. Example: gateway span, HTTP client span, DB query span.

Trace: abc123

```
Gateway      20ms
Order        80ms
Payment      900ms
DB           850ms
```

Total ≈ dominated by Payment/DB

10. Golden Signals

Production service ke liye four classic golden signals: latency, traffic, errors, saturation.

1. Latency
2. Traffic
3. Errors
4. Saturation

11. Latency

Request complete hone mein kitna time laga.

```
Request
  ↓
start
  ↓
...
  ↓
response
```

`duration = end - start`

12. p50 / Median

p50 ka matlab roughly 50% requests is latency se faster/equal hain aur 50% slower/equal.

100 requests sorted

p50 = middle area

Useful:
"typical" user experience

13. p95

p95 woh latency boundary hai jiske neeche approximately 95% observations hain. Remaining ~5% slower requests ko represent karta hai.

Example: p95 = 300 ms → approximately 95% requests 300 ms ya usse kam, while ~5% are slower.

14. p99

p99 tail latency ko represent karta hai. Approximately 99% requests threshold ke andar aur ~1% slower.

High-scale systems mein p99 important hai because average latency slow tail ko hide kar sakti hai.

15. Average vs Percentiles

Requests:
50ms
50ms
50ms
50ms
5000ms

Average ≈ 1040ms

But:
p50 ≈ 50ms
tail ≈ very slow

Average alone production user experience ko accurately represent nahi karta.

16. RPS / Throughput

RPS = requests per second. Traffic load ko understand karne ke liye key metric.

1 minute:

6000 requests

$RPS = 6000 / 60$
 $= 100 \text{ RPS}$

17. Error Rate

Error Rate =
 $\text{errors} / \text{total requests} \times 100$

Example:
 $500 \text{ errors} / 100,000 \text{ requests}$
 $= 0.5\%$

4xx and 5xx ko often separately monitor karo because client errors aur server failures ka meaning different hai.

18. Saturation

Saturation ka matlab resource capacity ke close ya overloaded hona. CPU, memory, DB connections, queue backlog, thread/worker pool etc. saturation indicators ho sakte hain.

CPU → 95%
DB pool → exhausted
Queue → growing
Memory → near limit

= saturation signals

19. Node.js Event Loop

Node.js JavaScript execution event loop par depend karta hai. CPU-heavy synchronous work event loop ko block kar sakta hai.

Request 1
↓
CPU-heavy sync loop
↓
Event loop blocked
↓
Request 2 waits
Request 3 waits
Request 4 waits

20. Event Loop Lag

Event loop lag roughly indicate karta hai ki scheduled work expected time se kitna late execute ho raha hai. High lag means Node process busy/blocking ho sakta hai.

Normal:
event loop lag ≈ low

CPU-heavy code:
event loop lag ↑
latency ↑
throughput ↓

21. Node.js Bottleneck Debugging

High API latency
↓
Check p95/p99
↓
CPU high?
↓
Event loop lag?
↓
DB latency?
↓
Redis latency?
↓
External API?
↓
GC/memory?
↓
Find bottleneck

22. CPU High

CPU high ka matlab automatically 'add more servers' nahi. Pehle identify karo CPU kis operation mein spend ho raha hai.

- CPU profile.
- Synchronous loops.
- JSON serialization/parsing.
- Crypto/compression.
- Large data transformation.
- Unexpected high traffic.

23. Memory High

Node.js memory growth ke liye heap usage, garbage collection, object retention aur potential leaks investigate karo.

Memory ↑
↓
Heap used?
↓
GC behavior?
↓
Objects retained?
↓
Large caches?
↓
Memory leak?

24. DB Observability

- Query latency.
- Slow query count.
- Rows examined/scanned.
- Query errors.
- Connection pool usage.
- Lock waits.
- CPU/IO.
- Replication lag.

25. Connection Pool

Node instances: 10
Pool per instance: 20

Potential max connections:
 $10 \times 20 = 200$

DB supports:
100

Problem:
connection exhaustion

Pool size ko blindly increase karna DB ko aur overload kar sakta hai.

26. Redis Observability

- Command latency.
- Hit ratio.
- Memory usage.
- Evictions.
- Connected clients.
- Blocked clients.
- Hot keys.
- Errors/timeouts.

27. Queue Observability

Producer
↓
Queue
↓
Workers

Important:
queue depth
oldest message age
consumer lag
processing latency
retry count
DLQ count

28. Queue Depth vs Lag

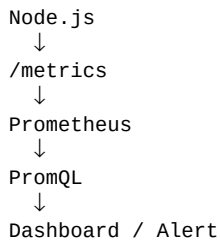
Queue depth current backlog batata hai. Consumer lag/time-to-process batata hai ki processing kitni behind hai. Dono useful but different signals hain.

29. External API Observability

- Request count.
- Latency p95/p99.
- Timeout rate.
- 5xx/4xx responses.
- Retry count.
- Circuit breaker state.

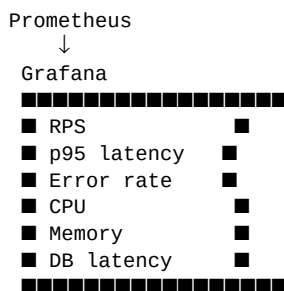
30. Prometheus

Prometheus metrics collect/store/query karne ke liye commonly used monitoring system hai. Applications metrics endpoint expose kar sakti hain aur Prometheus scrape kar sakta hai.



31. Grafana

Grafana metrics ko dashboards mein visualize karta hai. Prometheus common datasource hai, but Grafana multiple data sources ke saath kaam kar sakta hai.



32. PromQL Mental Model

```
rate(http_requests_total[5m])
```

Meaning:
recent 5-minute window mein
counter ka per-second rate

Exact PromQL production metric names and labels par depend karega.

33. Useful Dashboard

API Dashboard

Traffic:
RPS

Latency:
p50 / p95 / p99

Errors:
4xx / 5xx

Resources:
CPU / Memory / Event Loop Lag

Dependencies:
DB / Redis / Queue / External APIs

34. Alerting

Alert ka goal actionable incident signal dena hai, dashboard ka goal exploration/visibility. Har metric par alert banana alert fatigue create kar sakta hai.

Good alert:
5xx rate > threshold
AND sustained for X minutes

Bad:
CPU > 70% for 10 seconds
without user impact

35. SLI

SLI = Service Level Indicator. Actual measured reliability/performance signal.

SLI:
successful requests / total requests

or:
requests under 300ms / total requests

36. SLO

SLO = Service Level Objective. SLI ke liye target.

SLO:
99.9% requests successful

or:
99% requests < 300ms

37. SLA

SLA = customer/business-facing agreement with defined commitments and possible consequences. SLO aur SLA ko same thing mat samjho.

SLI → what we measure
SLO → internal target
SLA → external/business commitment

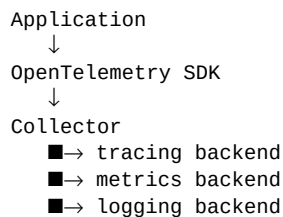
38. Error Budget

Agar SLO 99.9% hai, to allowed unreliability approximately 0.1% hai over the chosen period. Ye allowed budget release velocity vs reliability decisions mein help kar sakta hai.

SLO = 99.9%
Allowed error budget $\approx$ 0.1%

39. OpenTelemetry

OpenTelemetry instrumentation/telemetry collection ke liye vendor-neutral ecosystem hai. Traces, metrics aur logs ko standardized instrumentation pipeline ke through collect/export karne mein useful hai.



40. Correlation ID vs Trace ID

Correlation ID	Trace ID
Request/business correlation	Distributed trace identity
Useful in logs	Connects spans across services
Can be application-generated	Usually tracing system manages context

41. Production Debugging Example

Alert:

p99 latency ↑

Step 1:

Which endpoint?

Step 2:

Traffic increased?

Step 3:

CPU / Event Loop?

Step 4:

DB p99?

Step 5:

Redis?

Step 6:

External API?

Step 7:

Trace slow request

Step 8:

Identify bottleneck

Step 9:

Fix + verify metric recovery

42. Example: DB Bottleneck

API p99 ↑



Trace:

DB span = 900ms



EXPLAIN query



Full table scan



Add/adjust index



Re-test

↓
DB span = 30ms
↓
API p99 improves

43. Example: Node CPU Bottleneck

p99 ↑
CPU = 98%
Event Loop Lag = high
DB = normal
Redis = normal

Profile:
CPU-heavy synchronous function

Fix:
move heavy work to worker/queue
or optimize algorithm
or scale appropriately

44. Example: Queue Bottleneck

API latency normal
Async job completion slow

Queue depth ↑
Consumer lag ↑
Workers CPU = 95%

Action:
scale workers
check downstream limits
inspect failed/retry jobs

45. High RPS but Low CPU

Possible reasons: I/O-bound workload, waiting on DB/network, inefficient connection pool, external dependency latency or insufficient concurrency. CPU alone system health ka complete picture nahi deta.

46. Local Node.js + Prometheus + Grafana

```
node-app/  
  src/  
    server.js  
    metrics.js
```

Docker:
Node.js
Prometheus
Grafana
Redis
PostgreSQL

- Express API par request counter expose karo.
- Request duration histogram expose karo.
- Error counter add karo.
- CPU/memory/process metrics collect karo.
- Prometheus ko /metrics scrape karwao.
- Grafana dashboard create karo.
- Load generator se traffic generate karo.
- p50/p95/p99 observe karo.
- DB query intentionally slow karo.
- Trace/log se bottleneck locate karo.

47. Load Testing

```
Load Generator  
  ↓  
100 RPS  
  ↓  
Node API  
  ↓  
Prometheus  
  ↓  
Grafana
```

Increase:
100 → 200 → 500 → 1000 RPS

Observe:
latency
errors
CPU
DB
event loop

Load testing ka goal sirf maximum RPS find karna nahi; saturation point aur bottleneck identify karna bhi hai.

48. Capacity Thinking

Suppose one Node instance 100 RPS par p95 100ms deta hai. 3 instances theoretically more throughput provide kar sakte hain, but DB/Redis/network bottleneck ho sakta hai. End-to-end capacity weakest dependency se constrained hoti hai.

49. Interview Q&A;

Question	Short answer
Observability?	System behavior se internal state understand karne ki ability.

Three pillars?	Logs, metrics, traces.
p95?	Approximately 95% requests are at or below that latency.
p99?	Tail latency; approximately 99% at or below threshold.
RPS?	Requests per second.
Golden signals?	Latency, traffic, errors, saturation.
Event loop lag?	Node event loop execution delay indicator.
Prometheus?	Metrics collection/storage/querying and alerting ecosystem.
Grafana?	Dashboards/visualization and alerting UI.
OpenTelemetry?	Vendor-neutral telemetry instrumentation/collection ecosystem.
SLI?	Measured reliability/performance indicator.
SLO?	Target for an SLI.
SLA?	External/business service commitment.
How debug latency?	Endpoint → percentiles → resources → dependencies → trace → bottleneck.

50. Production Dashboard — 80/20

SERVICE OVERVIEW

Traffic:

RPS

Latency:

p50 | p95 | p99

Reliability:

4xx | 5xx | timeout

Node:

CPU | Memory | Event Loop Lag

DB:

Latency | Connections | Locks

Redis:

Latency | Hit Ratio | Memory

Queue:

Depth | Oldest Age | Consumer Lag

External APIs:

Latency | Errors | Timeouts

51. Final Golden Interview Answer

"First I establish whether the problem is traffic, latency, errors or saturation. I check p50/p95/p99, RPS and error rate, then drill into CPU, memory, Node event-loop lag and dependency metrics for DB, Redis, queues and external APIs. I use structured logs with request/trace IDs and distributed tracing to locate the slow span. After fixing the bottleneck, I verify the same production metrics improve rather than assuming the fix worked."

52. 80/20 Mental Model

USER COMPLAINS: "SYSTEM SLOW"

↓

1. Which endpoint?

↓

2. RPS increased?
↓
3. p50 / p95 / p99?
↓
4. Error rate?
↓
5. CPU / Memory?
↓
6. Node Event Loop?
↓
7. DB?
↓
8. Redis?
↓
9. Queue?
↓
10. External API?
↓
11. TRACE
↓
12. FIND BOTTLENECK
↓
13. FIX
↓
14. VERIFY METRICS